```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix,accuracy_score

pip install tensorflow
```

```
Requirement already satisfied: tensorflow in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (2.22.0rc0)
Requirement already satisfied: absl-py>=1.0.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (2.5.1)
Requirement already satisfied: astunparse>=1.6.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=25.9.23 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in
c:\users\user\appdata\local\python\pythoncore-3.14-64\lib\site-
packages (from tensorflow) (0.7.0)
Requirement already satisfied: google_pasta>=0.1.1 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (18.1.1)
Requirement already satisfied: opt_einsum>=2.3.2 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (3.4.0)
Requirement already satisfied: packaging in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from tensorflow)
(25.0)
Requirement already satisfied: protobuf<8.0.0,>=6.31.1 in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (6.33.5)
Requirement already satisfied: requests<3,>=2.21.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (2.32.5)
Requirement already satisfied: six>=1.12.0 in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from tensorflow)
(1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (3.3.0)
Requirement already satisfied: typing_extensions>=3.6.6 in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (4.15.0)
```

```
Requirement already satisfied: wrapt>=1.11.0 in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from tensorflow)
(2.5.0)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (1.84.0)
Requirement already satisfied: keras-nightly>=3.12.0.dev in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (3.16.0.dev2026101004)
Requirement already satisfied: numpy>=1.26.0 in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from tensorflow)
(2.3.4)
Requirement already satisfied: h5py~=3.15.1 in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from tensorflow)
(3.15.1)
Requirement already satisfied: ml_dtypes<1.0.0,>=0.5.1 in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
tensorflow) (0.6.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\user\appdata\
local\python\pythoncore-3.14-64\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
requests<3,>=2.21.0->tensorflow) (2025.10.5)
Requirement already satisfied: wheel<1.0,>=0.23.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
astunparse>=1.6.0->tensorflow) (0.48.0)
Requirement already satisfied: rich in c:\users\user\appdata\local\
python\pythoncore-3.14-64\lib\site-packages (from keras-
nightly>=3.12.0.dev->tensorflow) (15.0.0)
Requirement already satisfied: namex in c:\users\user\appdata\local\
python\pythoncore-3.14-64\lib\site-packages (from keras-
nightly>=3.12.0.dev->tensorflow) (0.1.0)
Requirement already satisfied: optree in c:\users\user\appdata\local\
python\pythoncore-3.14-64\lib\site-packages (from keras-
nightly>=3.12.0.dev->tensorflow) (0.20.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\user\
appdata\local\python\pythoncore-3.14-64\lib\site-packages (from rich-
>keras-nightly>=3.12.0.dev->tensorflow) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\
user\appdata\local\python\pythoncore-3.14-64\lib\site-packages (from
rich->keras-nightly>=3.12.0.dev->tensorflow) (2.19.2)
Requirement already satisfied: mdurl~=0.1 in c:\users\user\appdata\
```

```
local\python\pythoncore-3.14-64\lib\site-packages (from markdown-it-
py>=2.2.0->rich->keras-nightly>=3.12.0.dev->tensorflow) (0.1.2)
Note: you may need to restart the kernel to use updated packages.


[notice] A new release of pip is available: 25.3 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense ,Dropout

df=pd.read_csv("C:/Users/User/Downloads/churn_modeling.csv")

df.head()
```

```
   RowNumber  CustomerId   Surname  CreditScore Geography  Gender  Age
\
0          1    15634602  Hargrave          619    France  Female   42

1          2    15647311      Hill          608     Spain  Female   41

2          3    15619304      Onio          502    France  Female   42

3          4    15701354      Boni          699    France  Female   39

4          5    15737888  Mitchell          850     Spain  Female   43


   Tenure    Balance  NumOfProducts
0       2       0.00              1
1       1   83807.86              1
2       8  159660.80              3
3       1       0.00              2
4       2  125510.82              1
```

```
df.isnull()
```

```
      RowNumber  CustomerId  Surname  CreditScore  Geography  Gender
Age  \
0         False       False    False        False      False   False
False
1         False       False    False        False      False   False
False
2         False       False    False        False      False   False
False
3         False       False    False        False      False   False
False
4         False       False    False        False      False   False
False
...         ...         ...      ...          ...        ...     ...
...
```

```
9995      False       False    False        False      False   False
False
9996      False       False    False        False      False   False
False
9997      False       False    False        False      False   False
False
9998      False       False    False        False      False   False
False
9999      False       False    False        False      False   False
False

      Tenure  Balance  NumOfProducts
0      False    False          False
1      False    False          False
2      False    False          False
3      False    False          False
4      False    False          False
...      ...      ...            ...
9995   False    False          False
9996   False    False          False
9997   False    False          False
9998   False    False          False
9999   False    False          False

[10000 rows x 10 columns]
```

```
df.isnull().sum()
```

```
RowNumber        0
CustomerId       0
Surname          0
CreditScore      0
Geography        0
Gender           0
Age              0
Tenure           0
Balance          0
NumOfProducts    0
dtype: int64
```

```
df.shape
```

```
(10000, 10)
```

```
df.columns
```

```
Index(['RowNumber', 'CustomerId', 'Surname', 'CreditScore',
'Geography',
       'Gender', 'Age', 'Tenure', 'Balance', 'NumOfProducts'],
      dtype='object')
```

```
X=df.iloc[:,3:9]# feature variable
y=df.iloc[:,9]# target variable

labelencoder_gender=LabelEncoder()
X['Gender']=labelencoder_gender.fit_transform(X['Gender'])

#One hot encoding
X=pd.get_dummies(X,columns=['Geography'],drop_first=True)

X_train,x_test,y_train ,y_test=train_test_split(X,y,test_size=0.2,rand
om_state=42)

#data normalization
sc=StandardScaler()
X_train=sc.fit_transform(X_train)
X_test=sc.transform(x_test)

x_train_df=pd.DataFrame(X_train,columns=X.columns)
print("sample normalized training data ")
display(x_train_df.head())
print("\n Mean of features after scaling:\n",x_train_df.mean())
print("\n standard deviation of features after scaling:\
n",x_train_df.std())
```

```
sample normalized training data
```

```
   CreditScore    Gender       Age    Tenure   Balance
Geography_Germany  \
0     0.356500  0.913248 -0.655786  0.345680 -1.218471          -
0.579467
1    -0.203898  0.913248  0.294938 -0.348369  0.696838
1.725723
2    -0.961472  0.913248 -1.416365 -0.695393  0.618629          -
0.579467
3    -0.940717 -1.094993 -1.131148  1.386753  0.953212          -
0.579467
4    -1.397337  0.913248  1.625953  1.386753  1.057449          -
0.579467

   Geography_Spain
0        -0.576388
1        -0.576388
2         1.734942
3        -0.576388
4        -0.576388


 Mean of features after scaling:
 CreditScore          5.435652e-16
Gender               2.842171e-17
Age                 -1.896261e-16
```

```
Tenure                6.661338e-17
Balance              -3.099743e-16
Geography_Germany    -4.263256e-17
Geography_Spain      -7.194245e-17
dtype: float64

 standard deviation of features after scaling:
 CreditScore           1.000063
Gender                1.000063
Age                   1.000063
Tenure                1.000063
Balance               1.000063
Geography_Germany     1.000063
Geography_Spain       1.000063
dtype: float64
```

```
model = Sequential([
    Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(4, activation='softmax')
])
```

```
C:\Users\User\AppData\Local\Python\pythoncore-3.14-64\Lib\site-
packages\keras\src\layers\core\dense.py:100: UserWarning: Do not pass
an `input_shape`/`input_dim` argument to a layer. When using
Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
y_train = y_train - 1
y_test = y_test - 1
```

```
model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

```
history=model.fit(X_train,y_train,epochs=50,batch_size=32,validation_d
ata=(X_test,y_test),verbose=1)
```

```
Epoch 1/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.6357 - loss:
0.8074 - val_accuracy: 0.6725 - val_loss: 0.7423
Epoch 2/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6715 - loss:
0.7343 - val_accuracy: 0.6770 - val_loss: 0.7385
Epoch 3/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6735 - loss:
0.7289 - val_accuracy: 0.6755 - val_loss: 0.7370
Epoch 4/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6754 - loss:
0.7256 - val_accuracy: 0.6730 - val_loss: 0.7359
```

```
Epoch 5/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6768 - loss:
0.7234 - val_accuracy: 0.6730 - val_loss: 0.7354
Epoch 6/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6760 - loss:
0.7217 - val_accuracy: 0.6755 - val_loss: 0.7354
Epoch 7/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6764 - loss:
0.7202 - val_accuracy: 0.6745 - val_loss: 0.7355
Epoch 8/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6770 - loss:
0.7192 - val_accuracy: 0.6750 - val_loss: 0.7356
Epoch 9/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6785 - loss:
0.7181 - val_accuracy: 0.6730 - val_loss: 0.7355
Epoch 10/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6794 - loss:
0.7171 - val_accuracy: 0.6745 - val_loss: 0.7358
Epoch 11/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6800 - loss:
0.7163 - val_accuracy: 0.6750 - val_loss: 0.7356
Epoch 12/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6812 - loss:
0.7155 - val_accuracy: 0.6760 - val_loss: 0.7356
Epoch 13/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6809 - loss:
0.7148 - val_accuracy: 0.6770 - val_loss: 0.7359
Epoch 14/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6814 - loss:
0.7141 - val_accuracy: 0.6775 - val_loss: 0.7362
Epoch 15/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6826 - loss:
0.7136 - val_accuracy: 0.6775 - val_loss: 0.7368
Epoch 16/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6830 - loss:
0.7128 - val_accuracy: 0.6775 - val_loss: 0.7371
Epoch 17/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6831 - loss:
0.7121 - val_accuracy: 0.6795 - val_loss: 0.7373
Epoch 18/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6831 - loss:
0.7115 - val_accuracy: 0.6800 - val_loss: 0.7376
Epoch 19/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6826 - loss:
0.7109 - val_accuracy: 0.6805 - val_loss: 0.7377
Epoch 20/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6817 - loss:
0.7104 - val_accuracy: 0.6780 - val_loss: 0.7382
Epoch 21/50
```

```
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6812 - loss:
0.7098 - val_accuracy: 0.6790 - val_loss: 0.7384
Epoch 22/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6819 - loss:
0.7092 - val_accuracy: 0.6805 - val_loss: 0.7387
Epoch 23/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6820 - loss:
0.7086 - val_accuracy: 0.6815 - val_loss: 0.7390
Epoch 24/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6820 - loss:
0.7081 - val_accuracy: 0.6815 - val_loss: 0.7393
Epoch 25/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6827 - loss:
0.7076 - val_accuracy: 0.6815 - val_loss: 0.7394
Epoch 26/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6825 - loss:
0.7071 - val_accuracy: 0.6815 - val_loss: 0.7396
Epoch 27/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6827 - loss:
0.7066 - val_accuracy: 0.6795 - val_loss: 0.7400
Epoch 28/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6833 - loss:
0.7061 - val_accuracy: 0.6805 - val_loss: 0.7402
Epoch 29/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6835 - loss:
0.7056 - val_accuracy: 0.6805 - val_loss: 0.7405
Epoch 30/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6841 - loss:
0.7052 - val_accuracy: 0.6800 - val_loss: 0.7408
Epoch 31/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6845 - loss:
0.7047 - val_accuracy: 0.6805 - val_loss: 0.7413
Epoch 32/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6852 - loss:
0.7043 - val_accuracy: 0.6810 - val_loss: 0.7416
Epoch 33/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6851 - loss:
0.7039 - val_accuracy: 0.6800 - val_loss: 0.7419
Epoch 34/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6854 - loss:
0.7033 - val_accuracy: 0.6790 - val_loss: 0.7423
Epoch 35/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6861 - loss:
0.7028 - val_accuracy: 0.6800 - val_loss: 0.7428
Epoch 36/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6861 - loss:
0.7024 - val_accuracy: 0.6790 - val_loss: 0.7431
Epoch 37/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6864 - loss:
```

```
0.7020 - val_accuracy: 0.6795 - val_loss: 0.7435
Epoch 38/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6859 - loss:
0.7014 - val_accuracy: 0.6795 - val_loss: 0.7441
Epoch 39/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6864 - loss:
0.7011 - val_accuracy: 0.6800 - val_loss: 0.7444
Epoch 40/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6862 - loss:
0.7007 - val_accuracy: 0.6785 - val_loss: 0.7450
Epoch 41/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.6860 - loss:
0.7002 - val_accuracy: 0.6785 - val_loss: 0.7454
Epoch 42/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6871 - loss:
0.6998 - val_accuracy: 0.6785 - val_loss: 0.7456
Epoch 43/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6860 - loss:
0.6994 - val_accuracy: 0.6780 - val_loss: 0.7462
Epoch 44/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6864 - loss:
0.6991 - val_accuracy: 0.6775 - val_loss: 0.7466
Epoch 45/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6862 - loss:
0.6988 - val_accuracy: 0.6785 - val_loss: 0.7469
Epoch 46/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6867 - loss:
0.6984 - val_accuracy: 0.6795 - val_loss: 0.7476
Epoch 47/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6860 - loss:
0.6981 - val_accuracy: 0.6780 - val_loss: 0.7481
Epoch 48/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6867 - loss:
0.6976 - val_accuracy: 0.6785 - val_loss: 0.7486
Epoch 49/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6870 - loss:
0.6974 - val_accuracy: 0.6785 - val_loss: 0.7489
Epoch 50/50
250/250 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step - accuracy: 0.6867 - loss:
0.6970 - val_accuracy: 0.6775 - val_loss: 0.7494

train_acc=history.history['accuracy']
val_acc=history.history['val_accuracy']
print("Accuracy per Epoch:\n")
for i in range(len(train_acc)):
    print(f"Epoch{i+1}: training accuracy=
{train_acc[i]*100:.2f},validation accuracy={val_acc[i]*100:.2f}%")

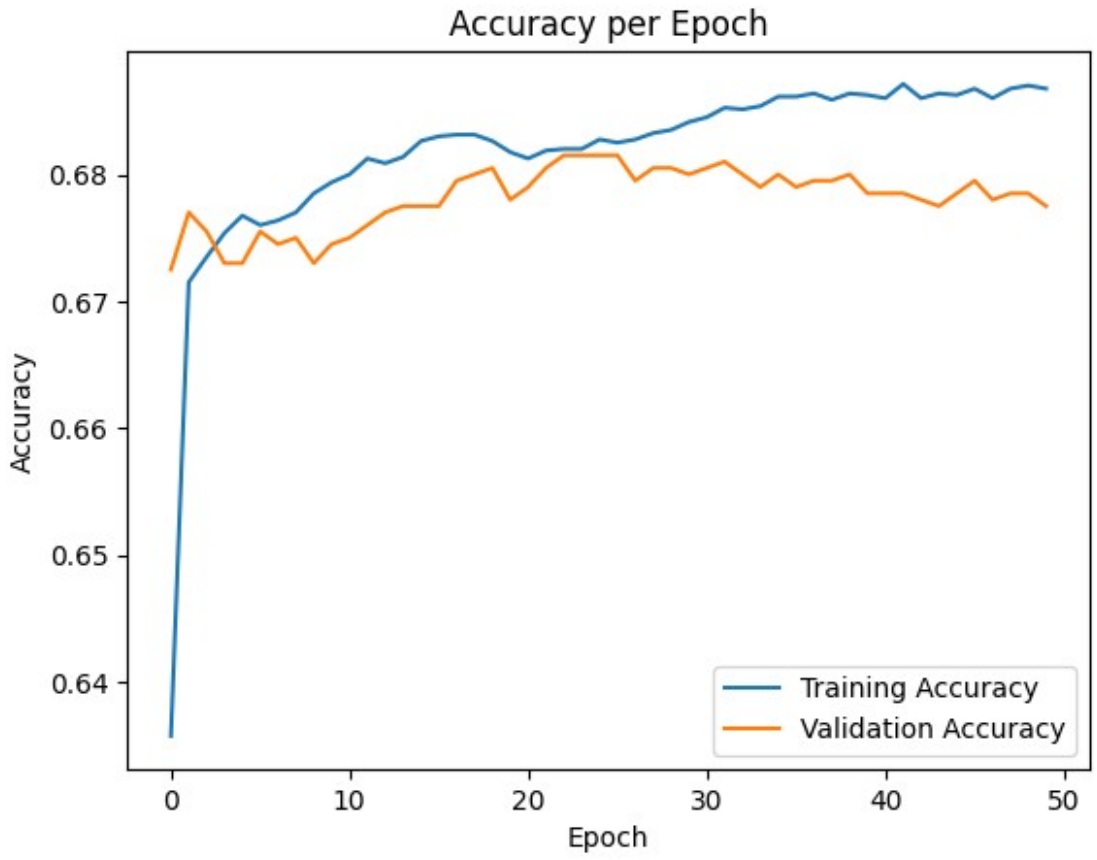
Accuracy per Epoch:
```

```
Epoch1: training accuracy= 63.57,validation accuracy=67.25%
Epoch2: training accuracy= 67.15,validation accuracy=67.70%
Epoch3: training accuracy= 67.35,validation accuracy=67.55%
Epoch4: training accuracy= 67.54,validation accuracy=67.30%
Epoch5: training accuracy= 67.68,validation accuracy=67.30%
Epoch6: training accuracy= 67.60,validation accuracy=67.55%
Epoch7: training accuracy= 67.64,validation accuracy=67.45%
Epoch8: training accuracy= 67.70,validation accuracy=67.50%
Epoch9: training accuracy= 67.85,validation accuracy=67.30%
Epoch10: training accuracy= 67.94,validation accuracy=67.45%
Epoch11: training accuracy= 68.00,validation accuracy=67.50%
Epoch12: training accuracy= 68.12,validation accuracy=67.60%
Epoch13: training accuracy= 68.09,validation accuracy=67.70%
Epoch14: training accuracy= 68.14,validation accuracy=67.75%
Epoch15: training accuracy= 68.26,validation accuracy=67.75%
Epoch16: training accuracy= 68.30,validation accuracy=67.75%
Epoch17: training accuracy= 68.31,validation accuracy=67.95%
Epoch18: training accuracy= 68.31,validation accuracy=68.00%
Epoch19: training accuracy= 68.26,validation accuracy=68.05%
Epoch20: training accuracy= 68.17,validation accuracy=67.80%
Epoch21: training accuracy= 68.12,validation accuracy=67.90%
Epoch22: training accuracy= 68.19,validation accuracy=68.05%
Epoch23: training accuracy= 68.20,validation accuracy=68.15%
Epoch24: training accuracy= 68.20,validation accuracy=68.15%
Epoch25: training accuracy= 68.27,validation accuracy=68.15%
Epoch26: training accuracy= 68.25,validation accuracy=68.15%
Epoch27: training accuracy= 68.27,validation accuracy=67.95%
Epoch28: training accuracy= 68.33,validation accuracy=68.05%
Epoch29: training accuracy= 68.35,validation accuracy=68.05%
Epoch30: training accuracy= 68.41,validation accuracy=68.00%
Epoch31: training accuracy= 68.45,validation accuracy=68.05%
Epoch32: training accuracy= 68.52,validation accuracy=68.10%
Epoch33: training accuracy= 68.51,validation accuracy=68.00%
Epoch34: training accuracy= 68.54,validation accuracy=67.90%
Epoch35: training accuracy= 68.61,validation accuracy=68.00%
Epoch36: training accuracy= 68.61,validation accuracy=67.90%
Epoch37: training accuracy= 68.64,validation accuracy=67.95%
Epoch38: training accuracy= 68.59,validation accuracy=67.95%
Epoch39: training accuracy= 68.64,validation accuracy=68.00%
Epoch40: training accuracy= 68.62,validation accuracy=67.85%
Epoch41: training accuracy= 68.60,validation accuracy=67.85%
Epoch42: training accuracy= 68.71,validation accuracy=67.85%
Epoch43: training accuracy= 68.60,validation accuracy=67.80%
Epoch44: training accuracy= 68.64,validation accuracy=67.75%
Epoch45: training accuracy= 68.62,validation accuracy=67.85%
Epoch46: training accuracy= 68.67,validation accuracy=67.95%
Epoch47: training accuracy= 68.60,validation accuracy=67.80%
Epoch48: training accuracy= 68.67,validation accuracy=67.85%
```

```
Epoch49: training accuracy= 68.70,validation accuracy=67.85%
Epoch50: training accuracy= 68.67,validation accuracy=67.75%
```

```
import matplotlib.pyplot as plt

# Accuracy per epoch
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Accuracy per Epoch')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```




```
y_pred_prob = model.predict(X_test)

y_pred = np.argmax(y_pred_prob, axis=1)
acc = accuracy_score(y_test, y_pred)

print("Final Accuracy: {:.2f}%".format(acc * 100))
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Final Accuracy: 67.75%
```

```
acc = accuracy_score(y_test, y_pred)
print("Final Accuracy: {:.2f}%".format(acc * 100))
```

```
Final Accuracy: 67.75%
```

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Confusion Matrix:
 [[803 229   0   0]
 [348 552   0   0]
 [ 29  24   0   0]
 [  9   6   0   0]]
```